

AdvanDEB Review — 2026-05-18

- [AdvanDEB — Codebase & Website Review](#)
 - [0. Executive Summary](#)
 - [1. Repository Map \(verified state on disk\)](#)
 - [2. Public website review — `https://advandeb.com`](#)
 - [3. Backend \(app/backend\) — code review](#)
 - [4. Frontend \(app/frontend\) — code review](#)
 - [5. knowledge-builder \(advandeb_kb\) — code review](#)
 - [6. mcp/ \(Rust gateway\) — code review](#)
 - [7. Documentation review](#)
 - [8. Operations & deployment](#)
 - [9. Security posture \(consolidated\)](#)
 - [10. Working-tree state \(uncommitted at review time\)](#)
 - [11. Plan of needed changes](#)
 - [12. Verification checklist for the next session](#)

AdvanDEB — Codebase & Website Review

Date: 2026-05-18 **Reviewer:** Claude (engineering review pass) **Scope:** app/, knowledge-builder/, mcp/, docs/, deployed site at `https://advandeb.com` **Branch reviewed:** main @ 2e3116f (with uncommitted working-tree changes — see §10)

0. Executive Summary

AdvanDEB is, on the inside, a substantive multi-component research platform: five Python ReAct agents behind a Rust MCP gateway, a FastAPI backend with WebSocket-streaming chat, a Vue 3 + Cosmograph KB visualizer, and a real ingestion pipeline that has already processed ~1300 papers and seeded 1236 stylized facts. The architecture documents (SYSTEM-OVERVIEW.md, ARCHITECTURE-REVISION.md, ROADMAP.md) are coherent, and the runtime topology in RUNNING.md matches what is on disk.

From the outside, almost none of that is visible. **The deployed site at `https://advandeb.com` presents only a Vue SPA shell behind a login wall.** An unauthenticated researcher landing on the domain sees a title, a sign-in form, and nothing else — no project description, no domain context, no example graphs, no docs, no contact, no SEO metadata. The platform is hidden behind auth even for read-only knowledge browsing, which contradicts the project's own goal of disseminating curated biology knowledge.

Inside the code there are also concrete defects to fix before any external launch:

- A **JWT type-confusion bug**: refresh tokens are accepted as access tokens on every protected route (auth.py vs core/auth.py — see §3.2).
- Two **broken test files** (tests/test_graph_service.py, tests/test_chat_service.py) reference modules/methods that no longer exist; the test suite does not collect cleanly.
- Configuration **drift** between core/config.py defaults and .env.example (MCP port 3000 vs 8080; OAuth redirect port 8000 vs 8400).
- **96 occurrences of `datetime.utcnow()`** — deprecated in Python 3.12 and scheduled for removal; the project pins Python 3.11 but conda environments may upgrade.
- **/health is shadowed by the static SPA** on production (returns index.html, not JSON) — production monitoring claims in README are effectively dead.

- **Several large godfiles** (1500+ lines): app/kb/pipeline.py (1523), knowledge-builder/services/graph_query_service.py (1625), agents at ~950 lines each.
- **No public-facing landing/marketing content, no SEO, no robots.txt semantics, no OpenGraph** — the site is invisible to search and to drive-by visitors.

None of this is fatal. The architectural backbone is sound and the data is real. What is missing is the last 10% that converts a working internal platform into a credible public research tool. The plan in §11 below lays out the change set in priority order.

1. Repository Map (verified state on disk)

advandeb/	monorepo
├── app/	main GUI (FastAPI + Vue 3)
│ ├── backend/ (Python 3.11)	
│ │ ├── app/main.py	FastAPI endpoint, lifespan, SPA mount
│ │ ├── app/core/	config, database, auth, dependencies
│ │ ├── app/api/routes/	auth, users, documents, facts, chat,
│ │ │ ├── app/api/routes/kb/	scenarios, models, ws, graph
│ │ │ │ ├── agents, database, ingestion, kg_builder,	
│ │ │ │ ├── visualization, visualization_stream,	
│ │ │ │ ├── documents, filesystem	
│ │ │ │ ├── chat_service, user_service,	
│ │ │ │ ├── user_submission_service,	
│ │ │ │ ├── provenance_service, scenario_service,	
│ │ │ │ └── model_service	
│ │ ├── app/clients/mcp_client.py	WebSocket JSON-RPC client → :8080/mcp
│ │ ├── app/kb/pipeline.py	1523 LOC – ingestion + KG-linking
│ │ ├── app/kb/watchdog.py	132 LOC – batch reconciliation
│ │ └── tests/	2 files, both currently broken (\$4)
│ └── frontend/ (Vue 3, TS, Vite, Pinia, Cosmograph)	
│ ├── src/views/	Home, Login, Chat, Documents, Facts,
│ │ ├── src/components/chat/	Scenarios, Models, KnowledgeBuilder
│ │ │ ├── ChatInterface, AgentActivity,	
│ │ │ ├── MessageList, MessageInput	
│ │ │ ├── CosmographCanvas, SchemaPanel,	
│ │ │ ├── NodeInspector, NodeTablePanel,	
│ │ │ └── FolderTree	
│ │ ├── src/components/documents/	DocumentLibrary, DocumentUpload
│ │ ├── src/components/provenance/	ProvenanceTrail
│ │ ├── src/stores/	auth, notifications
│ │ ├── src/workers/	graphArtifactWorker (web worker)
│ │ └── src/tests/	3 Vitest files (component + store)
├── knowledge-builder/ (installable: pip install -e .)	
│ ├── advandeb_kb/	
│ │ ├── agents/	base, react_engine (952), chatbot (946),
│ │ │ ├── curator, graph_explorer, query_planner,	
│ │ │ ├── retrieval, synthesis	
│ │ ├── services/	29 service files; graph_query_service
│ │ │ ├── (1625), chat_pipeline_service (554),	
│ │ │ ├── agent_framework (523), ingestion,	
│ │ │ ├── chunking, embedding, chromadb,	
│ │ │ ├── hybrid_retrieval, evidence_registry,	
│ │ │ ├── reference_verifier, taxonomy, ...	
│ │ ├── database/arango_client.py	ArangoDB sync client
│ │ ├── models/	Pydantic v2 models
│ │ └── tests/	6 unit test files
├── mcp/ (Rust 2021, Axum 0.7)	
│ ├── src/	2845 LOC, well-structured:
│ │ └── lib.rs (341)	router builder, AppState, /agents, /mcp

— gateway/	registry, router, pool, balancer, circuit_breaker, retry
— mcp/	protocol.rs, server.rs (WS handler)
— workflow/	context, executor (multi-step)
— ollama.rs, metrics.rs, config.rs	
— tests/	documented as "67 tests" in README
— docs/	
— SYSTEM-OVERVIEW.md	canonical component description
— ARCHITECTURE-REVISION.md	(referenced in memory; not re-read)
— CHATBOT-AGENT-SCHEME.md	302 LOC chatbot pipeline spec
— knowledge_graph_scheme_descriptions.md	graph schema reference
— ROADMAP.md	Phase 0..4 plan
— diagrams/	8 PlantUML diagrams w/ PNG+SVG
— scripts/	systemd units, start_all.sh, start_stack.sh, status_stack.sh, register_all_agents.sh
— papers/	~1300 source documents (data)
— data/, logs/, monitoring/, tests/	ops + integration smoke tests
— RUNNING.md	ops runbook (accurate)

2. Public website review — <https://advandeb.com>

The site is deployed (Cloudflare in front, cf - ray headers present) and the SPA loads. The findings below describe what an unauthenticated visitor sees.

2.1 What renders for an anonymous visitor

- Page title: AdvanDEB Modeling Assistant.
- Body before JS hydration: a single empty `<div id="app"></div>`.
- After hydration: `HomeView.vue` shows the H1 “*AdvanDEB Modeling Assistant*”, the line “*Main platform GUI for knowledge building and modeling*”, and a single blue **Login with Google** button. Nothing else.

That’s the entire public surface. There is no:

- Project mission, abstract, or DEB explainer
- Screenshots or example knowledge graphs
- “How it works” / pipeline diagram
- Roadmap or status
- Team / institution / contact
- Documentation links
- Public read-only graph or fact browser
- Footer with license, repository link, or imprint
- Cookie/consent notice (likely required if you ship to EU)

2.2 SEO, accessibility, performance

- **No `<meta name="description">`**, no OpenGraph (og:title, og:image), no Twitter cards, no `<link rel="canonical">`, no JSON-LD.
- **No SSR / no prerender** — Googlebot can usually render SPAs but Bing/DuckDuckGo and most academic-indexing bots cannot. For a research platform this is a discoverability problem.
- **No robots.txt content tested** (worth verifying).
- **No sitemap.xml**.
- **Favicon** is referenced but not verified.

- HTML response is small (<700 bytes before assets) which is fine, but the JS bundle and Cosmograph vendor chunks (cosmos-vendor-*.js) are loaded on every route, including login — wasted bandwidth for users who never see the graph.

2.3 Public endpoints exposed

- GET /api/auth/config → {"google_oauth_enabled": true} (public, OK).
- GET /health → **returns the SPA's index.html**, not JSON. The route is defined in main.py:171 but the production deployment (Cloudflare or upstream proxy) seems to route only /api/* to the FastAPI app and serve the SPA bundle for everything else. The /health endpoint advertised in RUNNING.md and app/README.md is therefore not reachable for external monitoring.
- GET /api/kb/viz/schemas → 401 (correctly requires auth).

2.4 Login / auth UX

LoginView.vue is a serviceable form (email/password + optional Google), but:

- The closing hint reads “*Don’t have an account? Contact an administrator.*” with no contact link, no email, no signup CTA.
- The Google client ID is read from import.meta.env.VITE_GOOGLE_CLIENT_ID. If that build-time env was not set, the button is shown but the OAuth URL is malformed (client_id=undefined). There is no runtime guard.
- After a failed Google callback the user sees the error inline, but on success the page navigates to / without clearing the ?code= query parameter from history — refresh re-submits the (now consumed) code.

2.5 Branding & visual design

The styling is the bare Vite scaffold (system font stack, hairline borders, inline CSS in each .vue file). There is no design system, no theme tokens, no dark mode. The Element Plus dependency is listed in package.json but LoginView and HomeView do not use it — usage is ad-hoc per-component.

3. Backend (app/backend) — code review

3.1 What’s good

- Clear separation: core/ (cross-cutting), api/routes/ (HTTP), services/ (business logic), clients/ (external), kb/ (pipeline).
- core/database.py cleanly documents the three data stores (app MongoDB, KB MongoDB workflow metadata, ArangoDB knowledge).
- Indexes are created idempotently in ensure_app_indexes().
- Background-task singleton election (_is_primary_worker) is at least thought about — comments are honest about its fragility.
- WebSocket chat (/ws/chat/{session_id}) handles JWT-in-query-param cleanly, including the 4401 close-code convention so the browser sees a structured failure.
- File logging via RotatingFileHandler is configured early.

3.2 Bugs and defects

#	Severity	Location	Issue
B1	High	core/auth.py:39-49 + get_current_user	verify_token does not check type. A refresh token therefore passes

#	Severity	Location	Issue
			Depends(<code>get_current_user</code>) and authenticates any protected endpoint. Only <code>/api/auth/refresh</code> checks <code>payload["type"] == "refresh"</code> . Mitigation: enforce <code>payload.get("type") != "refresh"</code> in <code>get_current_user</code> (and ideally tag access tokens with <code>type: "access"</code> explicitly).
B2	High	<code>core/auth.py:66</code>	<code>ObjectId(user_id)</code> is unguarded — a JWT with a malformed sub raises <code>bson.errors.InvalidId</code> → FastAPI returns 500 instead of 401.
B3	High	<code>api/routes/auth.py:127-131</code>	The bare <code>except Exception as e</code> returns <code>f"Authentication failed: {str(e)}"</code> to the client. Leaks Google SDK internals on failure.
B4	High	<code>services/user_service.py:98-112</code>	Timing-oracle: <code>authenticate_user</code> returns immediately if the user is absent (return <code>None</code>), but performs <code>bcrypt</code> verification if present. Response time leaks user existence. Always run <code>bcrypt</code> against a dummy hash on miss.
B5	High	<code>core/config.py:56</code> vs <code>.env.example:40</code>	<code>MCP_SERVER_URL</code> default is <code>http://localhost:3000</code> in code but <code>.env.example</code> sets it to <code>8080</code> . If <code>.env</code> is missing the var, the backend talks to the wrong port.
B6	High	<code>core/config.py:32-36</code> vs <code>.env.example:23</code>	<code>GOOGLE_REDIRECT_URI</code> example points to <code>localhost:8000</code> , but the backend runs on <code>8400</code> . Same drift as B5.
B7	Medium	<code>tests/test_graph_service.py</code>	Imports <code>app.services.graph_service.GraphService</code> — that module has been deleted (only stale <code>.pyc</code> files remain). Test fails on collection.
B8	Medium	<code>tests/test_chat_service.py</code>	Patches <code>svc._get_mcp_response</code> and expects the string <code>"placeholder"</code> for <code>MCP_SERVER_ENABLED=False</code> . The current <code>send_message</code> has no such method and returns <code>"MCP server is not enabled."</code> . Tests are stale by ≥1 refactor.
B9	Medium	<code>main.py:199</code>	The catch-all <code>@app.get("/{full_path:path}")</code> shadows <code>/health</code> in production deployments that don't whitelist <code>/health</code> separately at the proxy. <code>/health</code> is essentially dead in prod.
B10	Medium	<code>services/chat_service.py:34</code>	Hard-coded <code>_CHATBOT_AGENT_WS = "ws://localhost:8086"</code> . Should be <code>settings.CHATBOT_AGENT_WS</code> . Couples the app to a single-host deployment.
B11	Medium	<code>clients/mcp_client.py</code>	A new WebSocket connection is opened for every call (<code>call_tool</code> , <code>stream_tool_call</code>). Per-request connect + handshake under chat load. No reuse.
B12	Medium	<code>core/auth.py</code> + <code>services/*</code>	96 occurrences of <code>datetime.utcnow()</code> across <code>app/</code> and <code>knowledge-builder/</code> . Deprecated in Python 3.12 (DeprecationWarning:

#	Severity	Location	Issue
			<code>datetime.utcnow()</code> is deprecated). Should be <code>datetime.now(timezone.utc)</code> .
B13	Low	<code>api/routes/auth.py:181-184</code>	<code>/logout</code> is a no-op stub with a TODO; tokens remain valid until expiry (1h access / 30d refresh). For a research portal that's acceptable, but document it.
B14	Low	<code>api/routes/documents.py:46-54</code>	<code>POST /documents/upload</code> accepts <code>UploadFile</code> with no size cap or MIME validation at the route. Pushed entirely into <code>UserSubmissionService</code> — needs verification that bounds are enforced there.
B15	Low	<code>main.py:130</code>	<code>ProxyHeadersMiddleware(trusted_hosts="*")</code> trusts forwarded headers from anywhere. Fine while the only ingress is your own <code>nginx/Cloudflare</code> , but worth narrowing to the proxy IPs.
B16	Low	<code>services/user_service.py:152-173</code>	<code>assign_role / remove_role</code> skip the role-name vocabulary check — any string can be written into <code>users.roles</code> . Add a whitelist.

3.3 Architecture concerns

- **`app/kb/pipeline.py` is 1523 lines** — covers PDF jobs, KG linking, GPU-aware concurrency estimation, Ollama probing, and ChromaDB embedding. It should be split: `pipeline/concurrency.py`, `pipeline/pdf_job.py`, `pipeline/kg_link.py`, with thin glue.
- **Sync ArangoDB driver wrapped in a global 2-worker thread pool** (`visualization.py:23`, `provenance_service.py:18`). Under concurrent KB users this is the dominant bottleneck. Either:
 - Move to `python-arango-async` (preview), or
 - Raise the pool size and audit thread-safety of the singleton client.
- **Settings lacks production guardrails** — no validator on `JWT_SECRET_KEY` to reject the default "your-secret-key-here-..." string when `ENVIRONMENT != "development"`. A misconfigured deploy silently runs with the example secret.
- **CORS** is correctly origin-listed, but `allow_methods=["*"]`, `allow_headers=["*"]`, `allow_credentials=True` is permissive — fine for now, but credential-bearing wildcard methods/headers should be narrowed.
- **Worker-id heuristic** (`_is_primary_worker`) acknowledges in its own docstring that it's "an approximation". With the current single `uvicorn` process this is moot, but if you ever move to `gunicorn` workers the rebuild queue could double-fire or never fire. Use a real lease in MongoDB (a `singleton_locks` collection with TTL) instead.

3.4 Test coverage

- 2 backend test files, both broken (B7, B8). Effective backend coverage through `pytest` is currently zero.
- `knowledge-builder/tests/` has 6 files (models, taxonomy, chat models, agent pipeline, ingestion, `sf_seed`) — not exercised in this review, but they at least import current modules.
- `tests/integration/test_smoke.py` exists at repo root — worth verifying it still runs against the live stack.
- No CI configuration is present in `.github/`, `.gitlab-ci.yml`, etc. (none found).

4. Frontend (app/frontend) — code review

4.1 What's good

- Vue 3 Composition API + `<script setup>` throughout; idiomatic.
- Pinia store for auth with localStorage hydration + axios interceptor for transparent 401 → refresh-token → retry.
- `App.vue` correctly switches between an `AppLayout` wrapper and a bare `<router-view>` for login/KB, so the KB view can take the whole viewport.
- Cosmograph WebGL canvas is split into a dedicated component with a web-worker artifact loader (`workers/graphArtifactWorker.ts`).
- Vitest setup is in place with three component/store tests.

4.2 Issues

#	Severity	Location	Issue
F1	High	<code>views/HomeView.vue</code>	Marketing void. The home page for an authenticated user is a 6-card link grid; for an anonymous user it's literally <code><h1></code> + <code><p></code> + a Google login button. No project intro. VITE_GOOGLE_CLIENT_ID is read with no runtime null-check. If the env var was missing at build time the button silently builds <code>client_id=undefined</code> .
F2	High	<code>views/LoginView.vue:75-89</code>	After the OAuth callback, the <code>?code=</code> query parameter is not cleared from <code>window.location</code> . If the user reloads, the code is re-submitted and the second attempt fails.
F3	High	<code>views/LoginView.vue:91-103</code>	The 401-retry path sets <code>_retried</code> on the original request, but <code>refreshAccessToken()</code> only redirects on failure — there is no surface for transient network failures to bubble back with a useful toast.
F4	Medium	<code>utils/api.ts:25-35</code>	Tokens are kept in <code>localStorage</code> . For a JWT-bearer SPA this is the common choice but it is XSS-exfiltratable. Either: move access token to memory only + refresh in an <code>httpOnly</code> cookie, or document the threat model and CSP that mitigates it.
F5	Medium	<code>stores/auth.ts</code>	

#	Severity	Location	Issue
F6	Medium	App.vue:22-24	isBareRoute matches /login and any path starting with /kb. The KB view runs without the sidebar layout — fine — but this also strips the toast notifications mount, so KB errors never reach the user via the global notifier.
F7	Medium	package.json	Both marked (markdown) and highlight.js are top-level dependencies. Bundle size impact unverified; the README claims 645 KB gzipped but no analyzer output is committed.
F8	Low	views/DocumentsView.vue, components/kb/CosmographCanvas.vue	Two console.log calls remain (only 2 across the whole src/, so cheap to remove).
F9	Low	global CSS	No design tokens. LoginView, HomeView use hand-picked hex (#4285f4, #d32f2f, #357ae8). Element Plus theming is unused.
F10	Low	router guard	KB_ROLES = ['administrator', 'knowledge_curator'] is duplicated implicitly elsewhere (KnowledgeBuilderView.vue shows admin-only “Reset DB”). Centralize into a permissions.ts.

4.3 Tests

3 Vitest files, all touching UI behavior. No E2E (Playwright is in devDependencies but no playwright.config.ts is referenced anywhere visible; tests directory hasn’t been verified). The app/README.md boasts of an E2E suite that isn’t currently runnable.

5. knowledge-builder (advandeb_kb) — code review

5.1 What’s good

- The agent layer has a real base_agent.py + react_engine.py core, not just a pile of one-off scripts.
- Services are functionally cohesive — each addresses a single responsibility (chunking, embedding, chromadb, evidence registry, reference verifier, taxonomy, ...).
- ArangoDB is the canonical KB store with MongoDB only for ingestion workflow state — this split is documented in app/backend/app/core/database.py and matches reality.

5.2 Issues

- **services/graph_query_service.py (1625 LOC)** and **agents/react_engine.py (952 LOC)** and **agents/chatbot_agent.py (946 LOC)** are godfiles. They are the heart of the system, which makes them the highest-risk files to refactor — but also the ones that most need it. Suggested splits:
 - `graph_query_service`: per-schema query builders + a thin facade.
 - `react_engine`: prompt construction, tool dispatch, trace serialization in separate modules.
 - `chatbot_agent`: server framing (WebSocket JSON-RPC) separate from pipeline logic.
- **Two different Settings objects**: `advandeb_kb.config.settings` and `app.core.config.settings`. They drift independently — e.g. `OLLAMA_BASE_URL` exists in both, and `OLLAMA_MODEL` exists only in the KB one but is read by `app/kb/pipeline.py:101`. There is no single source of truth.
- **knowledge-builder/README.md** is out of date — describes a standalone backend on port 8000, references a `setup.sh` that doesn't exist, and presents the package as a self-hosted product rather than an embedded library. Per `docs/SYSTEM-OVERVIEW.md`, KB is now a *toolkit*. Reconcile.
- **Five top-level test files** at the package root (`test_agents_basic.py`, `test_agent_server.py`, `test_embedding_pipeline.py`, `test_kg_builder.py`) plus a `tests/` directory. Consolidate into `tests/` so `pytest` collects them automatically.

5.3 Pipeline status

Memory note: 1306 ingestion jobs queued, ~100+ completed, 3 concurrent Ollama workers, `sf_support` graph rebuilt at 3373 nodes / 2243 edges. This is the live state at last record — re-verifying actual numbers should be one of the first acts of the next session.

6. mcp/ (Rust gateway) — code review

6.1 What's good

- 2845 LOC, well-modularized:
`gateway/{registry,router,pool,balancer,circuit_breaker,retry}.rs`,
`mcp/{protocol,server}.rs`, `workflow/{context,executor}.rs`.
- Documented Prometheus metrics, optional TLS via `rustls`, env-variable overrides via `ADVANDEB_MCP_*`.
- Round-robin balancing, connection pooling, circuit breaker, retry — the resilience primitives expected of a gateway are present.
- README documents the protocol, REST endpoints, and configuration comprehensively.

6.2 Issues

- **No auth on the gateway**. Documented as intentional (it's “internal background service”) but it binds to `0.0.0.0:8080` by default. Verify the production firewall actually drops external traffic to 8080. A bound-loopback default (`127.0.0.1:8080`) would be safer.
 - **README claims 67 tests** — not verified in this review. Worth a `cargo test` run as a regression baseline.
 - **Workflow context resolution** (`workflow/context.rs:157 LOC`, `executor.rs:269 LOC`) uses string templating (`{{step0.result}}`). Document the templating grammar — this is the kind of thing that silently breaks when types change.
 - **No graceful shutdown wiring visible in `serve()`** — `axum::serve` blocks forever. Production restarts will drop in-flight WebSocket sessions hard. Add `with_graceful_shutdown` listening on `SIGTERM`.
-

7. Documentation review

- **docs/SYSTEM-OVERVIEW.md** is well-written, current, and matches the on-disk architecture.
 - **docs/ROADMAP.md** still presents Phase 0 (Multi-Agent RAG-KG) as forward-looking work; in practice large parts of it are done. The roadmap needs a status pass: which slices shipped, which slipped, what is next quarter.
 - **docs/CHATBOT-AGENT-SCHEME.md** (302 LOC) — the chatbot specification. Verify it matches `agents/chatbot_agent.py` as it exists today (file was touched recently per the working-tree diff).
 - **docs/knowledge_graph_scheme_descriptions.md** — feedback memory flags this file as one that *must* be updated whenever graph schemas change. Schema changes touched several routes recently (see commit 2e3116f); spot-check that this doc still reflects reality.
 - **app/README.md** is the most aspirational document in the repo — it claims production readiness, performance numbers, and an E2E suite. Trim it to what is actually true, or move the aspirational parts into `ROADMAP.md`.
 - **app/DEPLOYMENT.md**, **app/MONITORING.md**, **app/CONTRIBUTING.md** — referenced but not reviewed in detail. They should be sanity-checked for the same drift.
 - **RUNNING.md** is accurate and the most useful operations document.
-

8. Operations & deployment

- **Systemd units** in `scripts/` (`advandeb.service`, `advandeb-stack.service`, `advandeb-kb.service`) — the KB service is decommissioned per memory; remove the unit file if not needed to avoid drift.
 - **scripts/start_all.sh** is solid: waits for gateway health, starts 6 agents, captures PIDs into `logs/*.pid`. The matching `stop_all.sh` exists. Good.
 - **Production routing** (Cloudflare → backend on `:8400`): confirm what upstream serves `/`, `/assets/*`, and `/health`. Today `/health` returns SPA HTML — that’s a monitoring hole.
 - **No robots.txt, no sitemap.xml** found in `app/frontend/public/` (assuming standard Vite layout; not exhaustively verified).
 - **No public status page** — Cloudflare’s HTTP status alone isn’t enough; `mcp/:8080/health` and the six agent health endpoints are not externally surfaced.
-

9. Security posture (consolidated)

#	Severity	Issue
S1	High	JWT type confusion: refresh tokens authenticate access endpoints (B1).
S2	High	OAuth error path leaks raw exception strings to the client (B3).
S3	High	Email enumeration via login timing oracle (B4).
S4	High	JWT_SECRET_KEY has no startup validator against the example value. A misconfigured deploy silently runs with "your-secret-key-here-change-in-production".
S5	Medium	Tokens in localStorage (F5) — XSS exposure.
S6	Medium	MCP gateway has no auth and binds to 0.0.0.0 (verify firewall).
S7	Medium	ProxyHeadersMiddleware(trusted_hosts="*") (B15).
S8	Medium	No rate limiting (RATE_LIMIT_ENABLED=False in .env.example); login is brute-forceable.

#	Severity	Issue
S9	Medium	/api/auth/logout is a no-op — refresh tokens valid for 30 days even after “logout”.
S10	Low	assign_role accepts arbitrary role strings (B16).
S11	Low	Document upload has no size/MIME validation at the route level (B14).
S12	Low	No CSRF token. Acceptable for a pure Bearer-token JSON API, but document the threat model.
S13	Low	No Content-Security-Policy headers from the backend.

10. Working-tree state (uncommitted at review time)

The `git status` at the start of the session shows ~40 modified files and several deletions:

- **Deleted but unstaged:** `app/backend/app/models/document.py`, `app/backend/app/models/fact.py`, `app/backend/app/services/document_service.py`, `app/backend/app/services/fact_service.py`. These are part of the ongoing migration to user-submission + ArangoDB-canonical split. The `.pyc` files in `app/backend/app/services/__pycache__` still reference `graph_service` and probably others — clean up.
- **Modified:** most route files, several services, both backend pipelines, all primary KB agents, and ~10 frontend components. This is a large in-flight refactor that hasn’t been broken into commits.
- **Recommendation:** commit in logical slices (auth fixes, KB pipeline refactor, frontend KB visualization changes) so review and rollback are tractable. Right now a `git revert` would be a blunt instrument.

11. Plan of needed changes

The plan is ordered by **risk × visibility**: things that are both externally observable and low-cost come first; deep refactors come last.

P0 — Ship-blockers (this week)

1. **Fix JWT type confusion (S1 / B1).** In `core/auth.py::verify_token`, after decoding, raise 401 if `payload.get("type") == "refresh"`. Tag new access tokens explicitly with `"type": "access"`.
2. **Fix the /health shadow (B9).** Either (a) move `/health` into a dedicated router that the upstream proxy whitelists, or (b) add `/api/health` as the canonical endpoint and point monitors at it.
3. **Reject the default JWT_SECRET_KEY outside development (S4).** Add a `@field_validator("JWT_SECRET_KEY")` in `Settings` that fails to construct if `ENVIRONMENT != "development"` and the value contains `"your-secret-key-here"` or is shorter than 32 chars.
4. **Stop leaking exception text in /api/auth/google (B3 / S2).** Catch specific exceptions; log details server-side; return a generic 401 to the client.
5. **Repair the test suite.** Delete or rewrite `tests/test_graph_service.py` (the module is gone) and update `tests/test_chat_service.py` to the current `ChatService` API. Both files must collect cleanly before any CI is added.
6. **Reconcile MCP_SERVER_URL and GOOGLE_REDIRECT_URI defaults (B5, B6).** Make `core/config.py` defaults match `.env.example` exactly, or move to `pydantic Field(...)` with no fallback so a missing env var fails loudly.
7. **Commit the working tree in logical slices (§10).** No new code lands cleanly on top of an uncommitted 40-file refactor.

P1 — Public site fitness (next two weeks)

8. **Make `https://advandeb.com` say what AdvanDEB is.** Add a real landing page above the auth wall:
 - One-paragraph mission (“AdvanDEB is a curated knowledge graph for Dynamic Energy Budget biology…”).
 - A static screenshot of the graph + a short caption.
 - A 3-step “How it works” (ingest → extract → review/explore).
 - Links: docs, GitHub (if/when public), contact.
 - Footer with imprint/license and the team / institution behind the project. This is a content task more than an engineering one — it does not need to be SSR’d; a richer `HomeView.vue` for unauthenticated users is enough. SSR can wait.
9. **Add SEO basics.** `<meta name="description">`, OpenGraph (`og:title`, `og:image`, `og:description`, `og:url`), Twitter card, `robots.txt`, an initial `sitemap.xml` that lists at least `/` and `/login`. Ship a 1200×630 social-share image.
10. **Public read-only KB browsing (deferred — recommend Phase 1).** Allow Knowledge Explorer access *without* login for a curated subset (published facts, the `sf_support` graph). This is the project’s stated goal in `SYSTEM-OVERVIEW.md` (“Knowledge Explorer — Browse published knowledge, read-only access”). Right now even read-only browsing requires auth. Either:
 - Implement an “anonymous” virtual role with Knowledge Explorer capabilities and a hard read-only guard, **or**
 - Provide a static “explore” page that loads a published graph artifact directly from CDN (no API auth needed).
11. **Fix the OAuth `?code=reload` trap (B-F3).** On successful login, `router.replace('/')` so the code disappears from history.
12. **Clear console logs in `DocumentsView.vue` and `CosmographCanvas.vue` (F8).** Two lines.

P2 — Hygiene & correctness (next month)

13. **Replace `datetime.utcnow()` everywhere (B12).** 96 call sites; mechanical change to `datetime.now(timezone.utc)`. Add a ruff rule `DTZ003` (or a flake8 plugin) to prevent regressions.
14. **OAuth2PasswordBearer + `tokenUrl="/api/auth/token"` is wrong** — that endpoint doesn’t exist. Either implement the password-grant endpoint at that URL or point `tokenUrl` at `/api/auth/login`.
15. **Constant-time login (B4).** Always run `bcrypt` against a dummy hash on user-not-found.
16. **`get_current_user` ObjectId guard (B2).**
17. **Move hard-coded chatbot WebSocket URL to settings (B10).**
18. **MCP client connection reuse (B11).** Long-lived WebSocket pool; one connection per worker instead of one per call.
19. **Validate the working role vocabulary in `assign_role` (B16).**
20. **Add real CI.** GitHub Actions (or equivalent) with three jobs: `pytest` (backend + KB), `vitest && vue-tsc` (frontend), `cargo test && cargo clippy --deny warnings (mcp)`. Run on every PR.
21. **Refresh `app/README.md`** — drop the “Production Ready” framing until §11.P1 is shipped; trim the unverified performance numbers; move aspirations to `ROADMAP.md`.
22. **Update `knowledge-builder/README.md`** to present the package as an embedded library, not a standalone backend. Remove dead `setup.sh` references.
23. **Reconcile `docs/ROADMAP.md`** — mark Phase 0 slices that shipped as done; add a current quarter section.
24. **Consolidate the two Settings objects** (`app` and `advandeb_kb`) — pick a single configuration surface or document precisely which keys belong where.

P3 — Architecture & scale (next quarter)

25. **Split godfiles.**
 - `app/kb/pipeline.py` (1523) → `pipeline/{concurrency,pdf,kg_link}.py`.
 - `advandeb_kb/services/graph_query_service.py` (1625) → per-schema modules + a facade.

- `advandeb_kb/agents/react_engine.py` (952) and `chatbot_agent.py` 946. → separate transport from policy.
- 26. **Adopt an async ArangoDB driver** or scale the existing thread pool past 2 workers and audit thread-safety.
- 27. **Singleton task election via a MongoDB lease** rather than the `_is_primary_worker` heuristic.
- 28. **Token storage hardening (S5):** access token in memory, refresh token in an `httpOnly + Secure + SameSite=Strict` cookie. Document the CSP that goes with it.
- 29. **Add a real /logout (S9)** — track issued refresh jtis in a `revoked_tokens` collection with TTL.
- 30. **Rate-limit the auth endpoints (S8)** — `slowapi` is a 50-line add for `POST /api/auth/login` and `/api/auth/google`.
- 31. **MCP gateway graceful shutdown** with `with_graceful_shutdown` on `SIGTERM` so deploys don't drop sessions hard.
- 32. **Bind MCP to loopback** by default; require an explicit env var to expose it externally.
- 33. **End-to-end Playwright suite** that covers: login, upload, KB graph load, chat with citation expansion, provenance trail. Three scenarios beat thirty unit tests for a system this integrated.

P4 — Strategic

- 34. **Decide the “public-readable” boundary.** The codebase has a Knowledge Explorer role but no anonymous path. Either commit to a public knowledge base (and design for it — caching, indexing, abuse handling) or remove the read-only role from the documentation.
- 35. **Decide whether the project is open source.** README sections reference a fork-and-PR workflow with no public repo; the integrations doc references “Dev 1/2/3” handoffs. Either publish the repo or update the docs to reflect the closed model.
- 36. **A real design system.** Pick Element Plus or Tailwind tokens, not both; build a tokens file; retire the hand-rolled hex codes in `LoginView/HomeView`.

12. Verification checklist for the next session

Before relying on any P1/P2 claim above, re-confirm:

- ☐ `git status` is clean (or at least understood) before edits start.
- ☐ `pytest` collects without errors in `app/backend/` and `knowledge-builder/`.
- ☐ `cargo test` and `cargo clippy --deny warnings` pass in `mcp/`.
- ☐ `curl https://advandeb.com/api/health` (after the §11.P0.2 fix) returns JSON, not HTML.
- ☐ `curl -H "Authorization: Bearer <refresh_token>" https://advandeb.com/api/users/me` returns 401 (after the §11.P0.1 fix), proving JWT type confusion is closed.
- ☐ Spot-check `docs/knowledge_graph_scheme_descriptions.md` against the live graph schemas in ArangoDB.
- ☐ `app/frontend/dist/index.html` after `npm run build` contains a `<meta name="description">` (proves §11.P1.9 is wired).